RCC LVGL Introduction and Source Code Analysis

1. Introduction

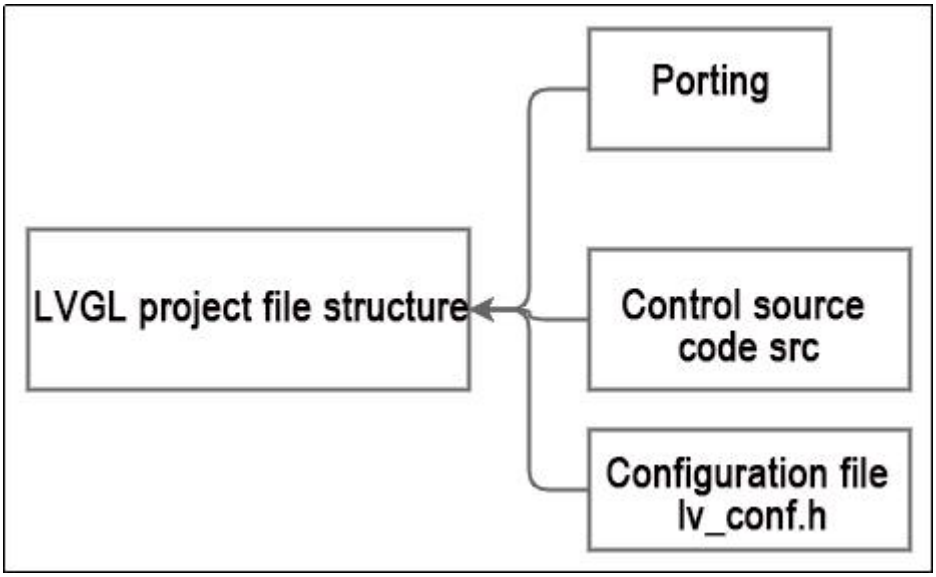
LVGL is a free and open-source graphics library, often considered a lightweight display framework, providing all the necessary features to create embedded graphical user interfaces, including easy-to-use graphical elements, good visual effects, and low memory consumption. It is written in C language, follows the MIT license, and can be freely used and modified. LVGL supports various operating systems such as Linux, Windows, RTOS, etc., and can also run on development boards. LVGL also supports various display drivers and touchscreen drivers, making it compatible with displays of different sizes and resolutions. LVGL (Graphics Library) features lightweight, multifunctional, flexible, and portable characteristics, and it is also compatible with the C++ programming language.



2. Source Code Analysis

2.1 Source Code Structure

According to the LVGL official website's downloaded archive (located at: "Lesson 19 RCC LVGL Introduction and Source Code Analysis/RCC LVGL Source Code and Porting Routines/Source Code and Projects to be Ported"), proceed with program design after porting.



2.2 Source Code Structure Analysis

The upcoming program analysis will be based on the "rosrobotcontrollerm4_LVGL" project file located in "Lesson 19 RCC LVGL Introduction and Source Code Analysis / Ported Example". Users can refer to it for comparison.

- Porting：

The "Porting" folder contains three C files and their corresponding header files.



Among them, lv_port_disp.c and its corresponding header file are commonly used, which contain the driver functions for driving the LVGL screen. The main functions to pay attention to inside are as follows:

lv_port_disp_init：

```
void lv_port_disp_init(void)
```

Selects the graphic buffer mode, whether to use one buffer, two buffers, or three buffers. Here, only one buffer is used.

Initializes the display driver structure.

```
/*-------------------------
 * Initialize your display
 * -----------------------*/
disp_init();
```

Sets the screen resolution size, for example, in this case, the screen is 80x160 pixels (width x height).

```
disp_drv.hor_res = 160;
disp_drv.ver_res = 80;
```

Specifies the refresh call function. Here, the default disp_flush() function from the official example is used. This function needs to be written and completed by the user, primarily for drawing functions on the screen.

```
/*Used to copy the buffer's content to the display*/
disp_drv.flush_cb = disp_flush;
```

Sets the screen display buffer mode. Here, draw_buf_dsc_1 is chosen for low power consumption. It's recommended to keep the default setting.

```
/* Example for 1) */
static lv_disp_draw_buf_t draw_buf_dsc_1;
static lv_color_t buf_1[MY_DISP_HOR_RES * 10];                          /*A buffer for 10 rows*/
lv_disp_draw_buf_init(&draw_buf_dsc_1, buf_1, NULL, MY_DISP_HOR_RES * 10);   /*Initialize the display buffer*/
```

Another important function is the refresh function (disp_flush):

```
/*Flush the content of the internal buffer the specific area on the display
 *You can use DMA or any hardware acceleration to do this operation in the background but
 *'lv_disp_flush_ready()' has to be called when finished.*/
static void disp_flush(lv_disp_drv_t * disp_drv, const lv_area_t * area, lv_color_t * color_p)
{
    /*The most simple case (but also the slowest) to put all pixels to the screen one-by-one*/

    int32_t x;
    int32_t y;
    for(y = area->y1; y <= area->y2; y++) {
        for(x = area->x1; x <= area->x2; x++) {
            /*Put a pixel to the display. For example:*/
            /*put_px(x, y, *color_p)*/
            color_p++;
        }
    }

    /*IMPORTANT!!!
     *Inform the graphics library that you are ready with the flushing*/
    lv_disp_flush_ready(disp_drv);
}
```

This function directly provides the screen drawing interface to LVGL, allowing for the direct scanning, refreshing, and filling of individual pixels on the screen.

The lv_port_fs file is related to the file system, providing functions for reading and writing to files. Users can refer to the comments associated with each function for understanding and learning. These functions mainly include file opening, reading, writing, and closing operations.

```
static void * fs_open(lv_fs_drv_t * drv, const char * path, lv_fs_mode_t mode);
static lv_fs_res_t fs_close(lv_fs_drv_t * drv, void * file_p);
static lv_fs_res_t fs_read(lv_fs_drv_t * drv, void * file_p, void * buf, uint32_t btr, uint32_t * br);
static lv_fs_res_t fs_write(lv_fs_drv_t * drv, void * file_p, const void * buf, uint32_t btw, uint32_t * bw);
static lv_fs_res_t fs_seek(lv_fs_drv_t * drv, void * file_p, uint32_t pos, lv_fs_whence_t whence);
static lv_fs_res_t fs_size(lv_fs_drv_t * drv, void * file_p, uint32_t * size_p);
static lv_fs_res_t fs_tell(lv_fs_drv_t * drv, void * file_p, uint32_t * pos_p);

static void * fs_dir_open(lv_fs_drv_t * drv, const char *path);
static lv_fs_res_t fs_dir_read(lv_fs_drv_t * drv, void * rddir_p, char * fn);
static lv_fs_res_t fs_dir_close(lv_fs_drv_t * drv, void * rddir_p);
```

The lv_port_indev file is related to touch screen drivers. When the screen has touch functionality, the functions inside can be called for use. For example:

```
static void touchpad_init(void);
static void touchpad_read(lv_indev_drv_t * indev_drv, lv_indev_data_t * data);
static bool touchpad_is_pressed(void);
static void touchpad_get_xy(lv_coord_t * x, lv_coord_t * y);
```

- touchpad_init(void): Initializes the touchpad of the screen.
- touchpad_read(): Reads touch signals.

- touchpad_is_pressed(void): Reads press signals.
- touchpad_get_xy(): Retrieves the x and y coordinates of the touch position.

```
static void mouse_init(void);
static void mouse_read(lv_indev_drv_t * indev_drv, lv_indev_data_t * data);
static bool mouse_is_pressed(void);
static void mouse_get_xy(lv_coord_t * x, lv_coord_t * y);

static void keypad_init(void);
static void keypad_read(lv_indev_drv_t * indev_drv, lv_indev_data_t * data);
static uint32_t keypad_get_key(void);

static void encoder_init(void);
static void encoder_read(lv_indev_drv_t * indev_drv, lv_indev_data_t * data);
static void encoder_handler(void);
```

Similarly, the file includes functions for mouse and keyboard operations.

- Control source code src file:



The src folder contains many source files of the LVGL library, including:

- core: This folder contains core files of LVGL, covering many built-in initialization and driver settings. Users can call corresponding functions according to their needs.
- draw: This folder deals with screen font style settings. When users need to

adjust the style of their interface, they can refer to the functions in this folder for customization.

- extra: This folder contains complex widgets created from other widgets, or modifications to existing widgets with special events, styles, or animations. Users can explore the contents for additional functionalities that can be used as LVGL plugins.
- font: This folder contains various font styles for users to customize and beautify their interfaces. It's important to choose fonts based on the memory size of the screen.
- gpu: When the screen has a GPU driver, the contents of this folder can be called for settings.
- hal: This folder provides hardware-accelerated interfaces for GPU acceleration. Users can refer to implementations in lv_port_disp.c for CPU-based hardware acceleration.
- misc: This folder contains miscellaneous functionalities, such as image rotation and stretching. Users can explore the source files in this folder for function calls related to image styling.
- Widgets: This folder contains pre-configured widget styles, such as sliders, buttons, and text areas.
- Configuration file lv_conf.h：

This file contains macro definitions related to screen configuration, used for settings.

1. Define maximum LCD screen resolution

LV_HOR_RES_MAX is used to set the horizontal length of the LCD screen, and LV_VER_RES_MAX is used to set the vertical length of the LCD screen.

2. Define color depth

The values of the LV_COLOR_DEPTH configuration item correspond to 1, 8, 16, 32, representing the formats as follows: 1 byte per pixel, RGB233, RGB565, ARGB8888, respectively. It is usually set to 16.

3. Enable anti-aliasing feature

There's only one configuration item related to this feature, which is LV_ANTIALIAS. Its default value is 1, meaning anti-aliasing is enabled by default. When set to 0, anti-aliasing is disabled. In practical projects, it's generally enabled, so that elements like text, lines, and arcs in our UI interface won't exhibit jagged edges.

4. Define the refresh period of the screen

The only related configuration item for this feature is LV_DISP_DEF_REFR_PERIOD, which has a default value of 30ms. If this value is set too large, it may lead to stuttering. On the other hand, setting it too small may waste performance. It's generally fine to stick with the default value without worrying too much.

5. Define the Polling Period of the Input Device

This function is associated with 1 configuration item, which is LV_INDEV_DEF_READ_PERIOD. Its default value is 30ms, meaning that the input device (such as a keyboard, touchscreen, etc.) is sampled every 30ms to check its status.

6. Definition of Drag Threshold

Associated with this feature is 1 configuration item, LV_INDEV_DEF_DRAG_LIMIT, with a default value of 10 pixels. This means that dragging more than 10 pixels while holding down is considered a valid drag action.

7. Definition of Long Press Event Trigger Time

The relevant configuration item for this feature is LV_INDEV_DEF_LONG_PRESS_TIME, with a default value of 400ms. This indicates that holding down a control for more than 400ms will trigger its long press event.

8. Definition of Long Press Repeat Event Trigger Time

The related configuration item for this feature is LV_INDEV_DEF_LONG_PRESS_REP_TIME, with a default value of 100ms. This means that after triggering a control's long press event, holding it down for more than 100ms will trigger its long press repeat event.

9. Enablement of Image Alpha Transparency Feature

Associated with this feature is 1 configuration item, LV_IMG_CF_ALPHA, with a default value of 1, indicating the enabling of alpha transparency feature. A value of 0 indicates disabling.

10. Explicit Definition of Memory Alignment Size

The configuration item LV_ATTRIBUTE_MEM_ALIGN is related to this feature, with a default value of empty. This is because when the compiler optimizes using -Os, it may result in our data not being aligned to 4 or 8 bytes. In such cases, we can explicitly define byte alignment.

11. Enablement of LittlevGL's Built-in Themes

The related configuration items for this feature are 9 in total, namely:

LV_THEME_LIVE_UPDATE,
LV_USE_THEME_TEMPL,
LV_USE_THEME_DEFAULT,

```
LV_USE_THEME_ALIEN,

LV_USE_THEME_NIGHT,

LV_USE_THEME_MONO,

LV_USE_THEME_MATERIAL,

LV_USE_THEME_ZEN,

LV_USE_THEME_NEMO
```

In which, LV_THEME_LIVE_UPDATE is used to determine whether to allow switching theme styles during runtime, while the other 8 configuration options indicate whether to enable the corresponding 8 theme styles.

12. Enable Built-in Fonts in LittlevGL

```
LV_FONT_ROBOTO_12,

LV_FONT_ROBOTO_16,

LV_FONT_ROBOTO_22,

LV_FONT_ROBOTO_28,

LV_FONT_UNSCII_8
```

There are 5 related configuration options for this feature, corresponding to Roboto fonts in sizes 12, 16, 22, and 28, and the Unscii font in size 8. Each font includes ASCII characters and some icon fonts (Symbols), all of which are 4 bits per pixel anti-aliased fonts. Setting to 1 enables them, while setting to 0 disables them.

13. Define the Text Encoding Method

There is one related configuration option, LV_TXT_ENC, with two possible values: LV_TXT_ENC_UTF8 (default) and LV_TXT_ENC_ASCII. When displaying Chinese or other non-ASCII characters, LV_TXT_ENC_UTF8 must

be set, and the code editor must also be set to the corresponding UTF-8 encoding.

3. Porting Method

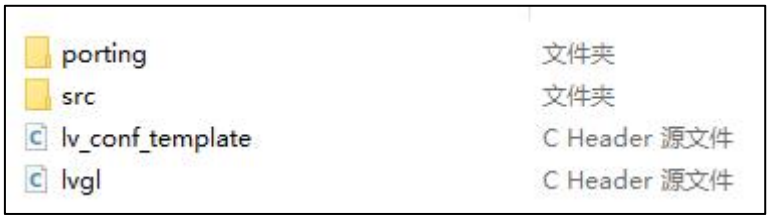
When users need to develop their own projects, they can follow the steps below to port lvgl into their project. The specific steps are as follows:

(1) Create a new folder named "Lvgl" in the newly created project file. The name can be customized.

Here, a project file is provided for users to simulate testing (**located in: "Lesson 19 RCC LVGL Introduction and Source Code AnalysisRCC LVGL Source Code and Porting Routines/Source Code and Projects to be Ported"**).

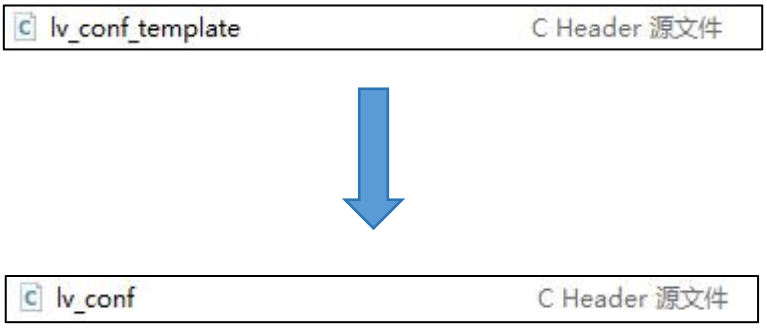


(2) Copy the 'src' folder, 'lv_conf_template', 'lvgl' files inside the 'lvgl_configuration' directory under the current directory 'Lesson 19 RCC LVGL Introduction and Source Code Analysis/RCC LVGL Source Code and Porting Routines/Source Code and Projects to be Ported', as well as the 'example/Porting' folder, into the 'Lvgl' folder as shown in the figure below:

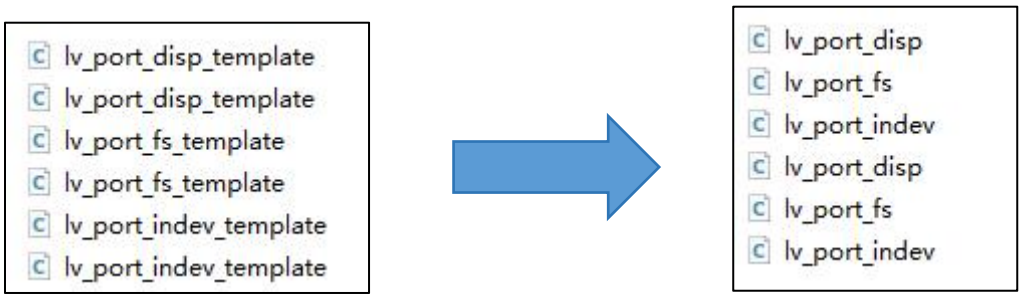


(3) Change the folder and folder name

- Rename the configuration file 'lv_conf_template.h' to 'lv_conf.h'.



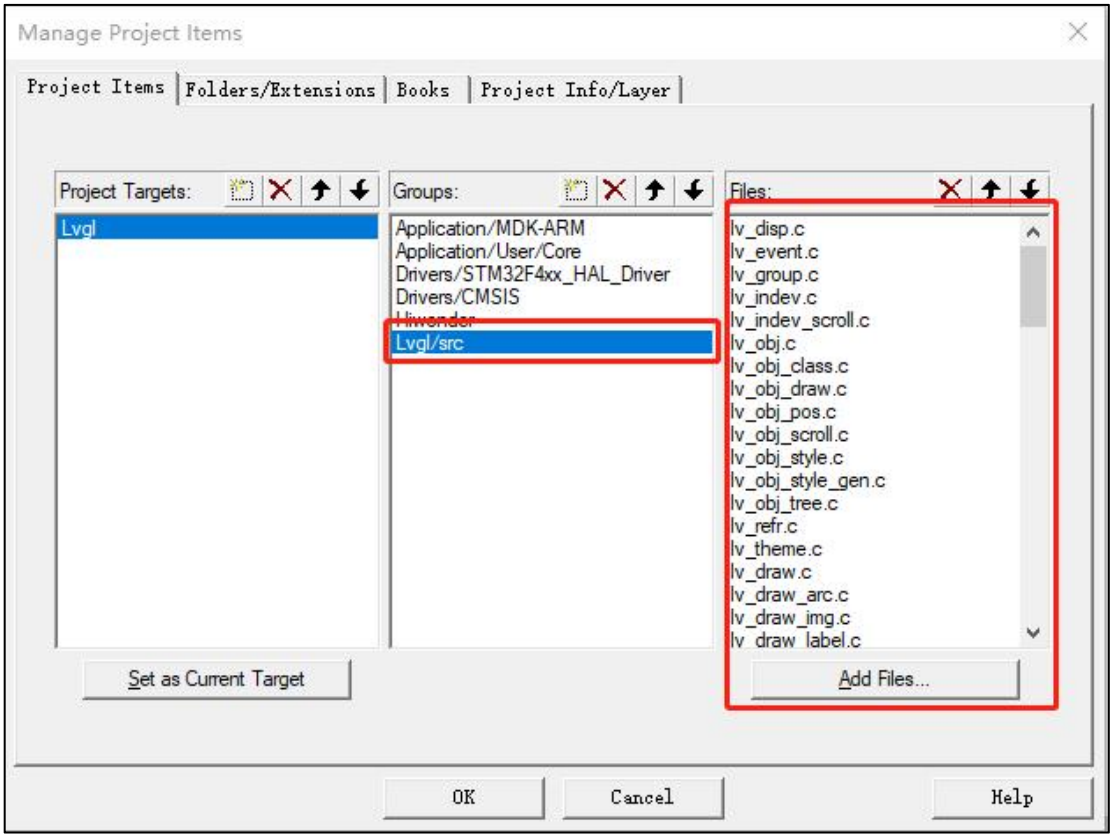
- Rename the files under 'examples/porting' as well, removing the '_template' suffix



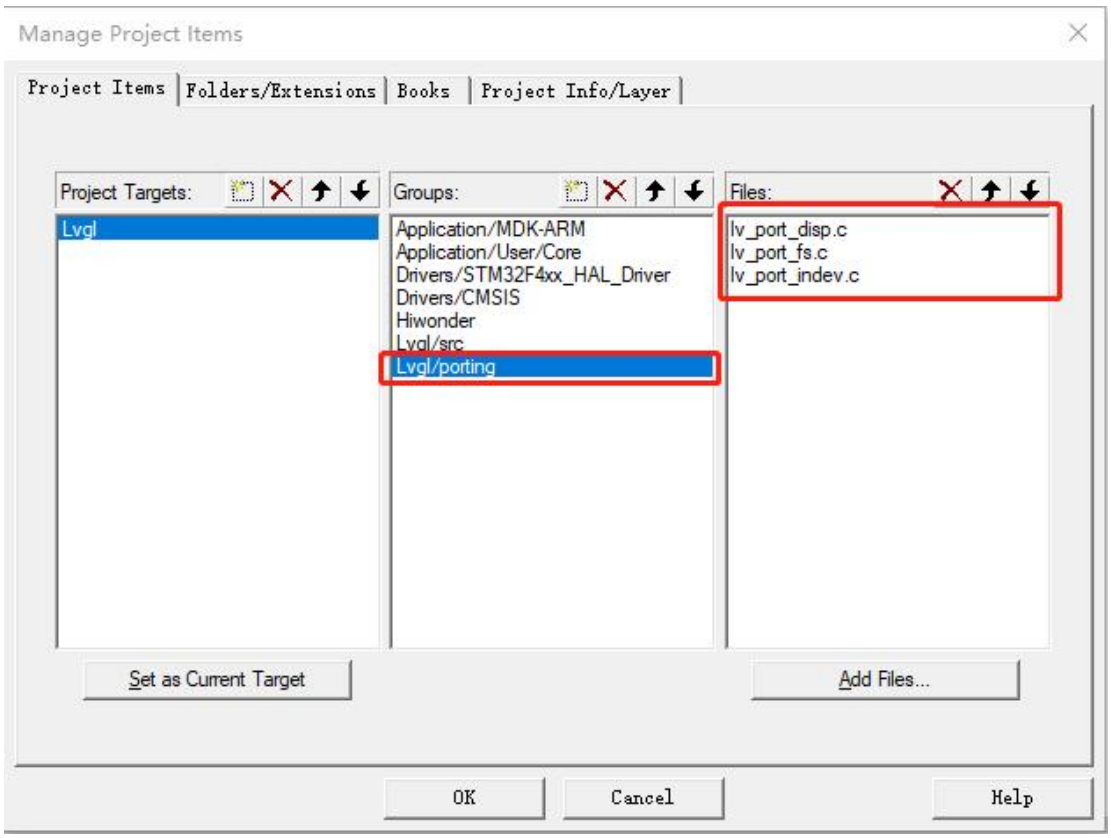
(4) **Adding Lvgl to the project:**

Open the project file with Keil5 (located in the MDK-ARM folder), then click on  to configure

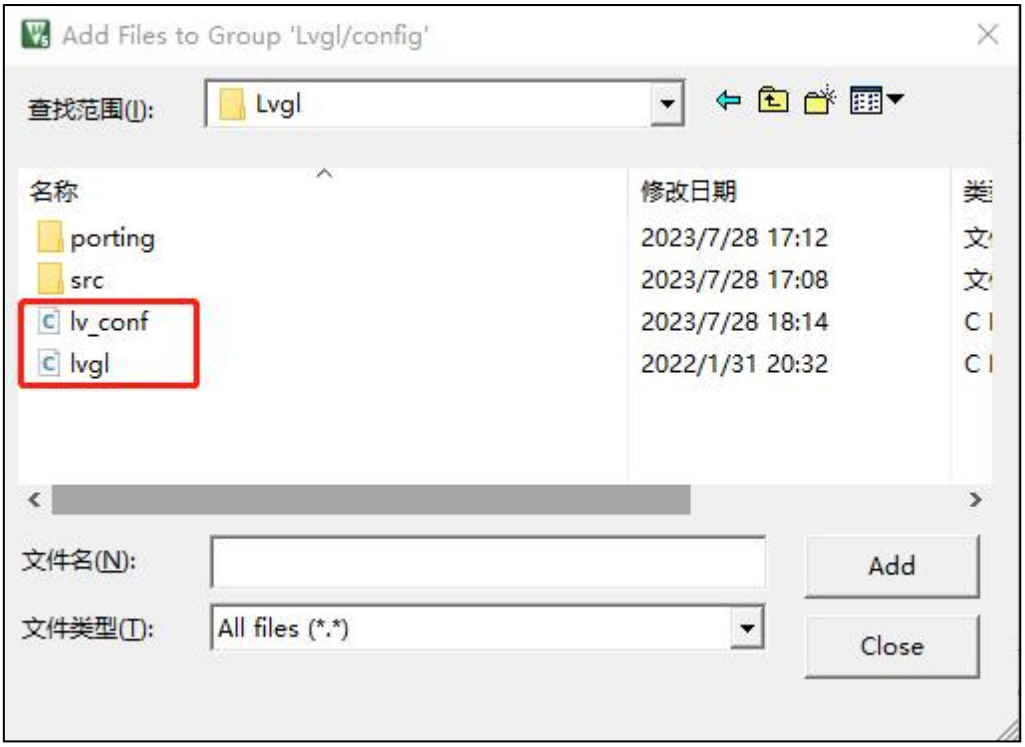
Add the Lvgl source code by including all the .c files from the 'src' directory into the project. Exclude the 'draw' folder from the 'src' directory, except for the contents of the 'sw' folder.

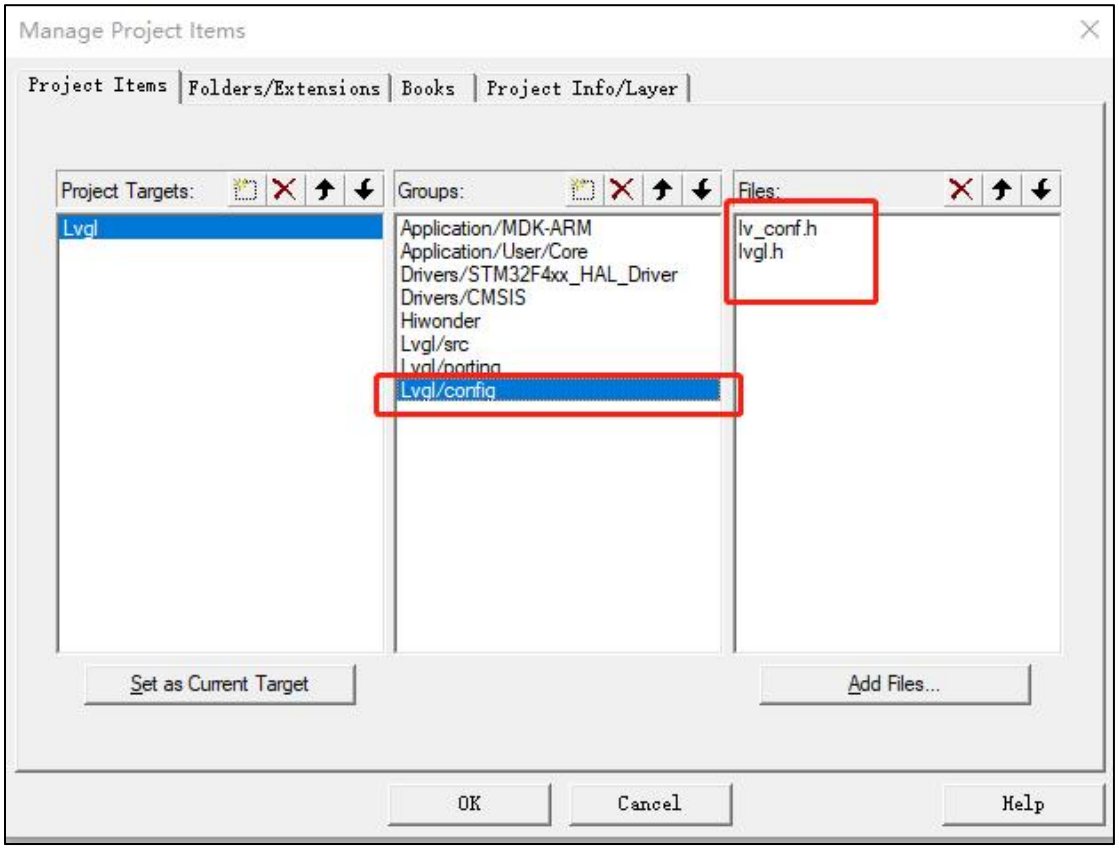


Also, add the source files from 'Lvgl/Porting' to the project:



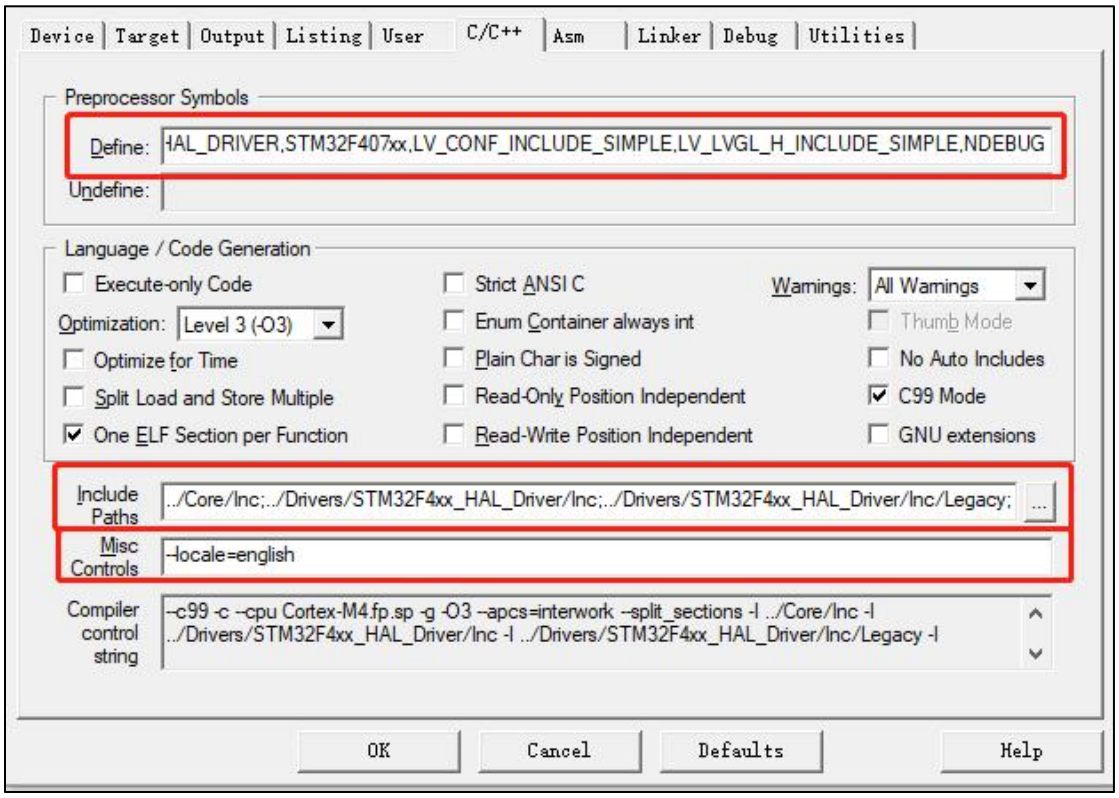
Then, include the configuration files. These two files are located inside the 'Lvgl' folder. When adding them, select 'all files' to view them.





(5) Include header files and relevant macro definitions, configure the compilation settings, and click on the magic wand icon at the top of the Keil5 software, as shown in the figure below:





Set macro definitions by filling in the following content in the 'Define' section, separated by English commas:

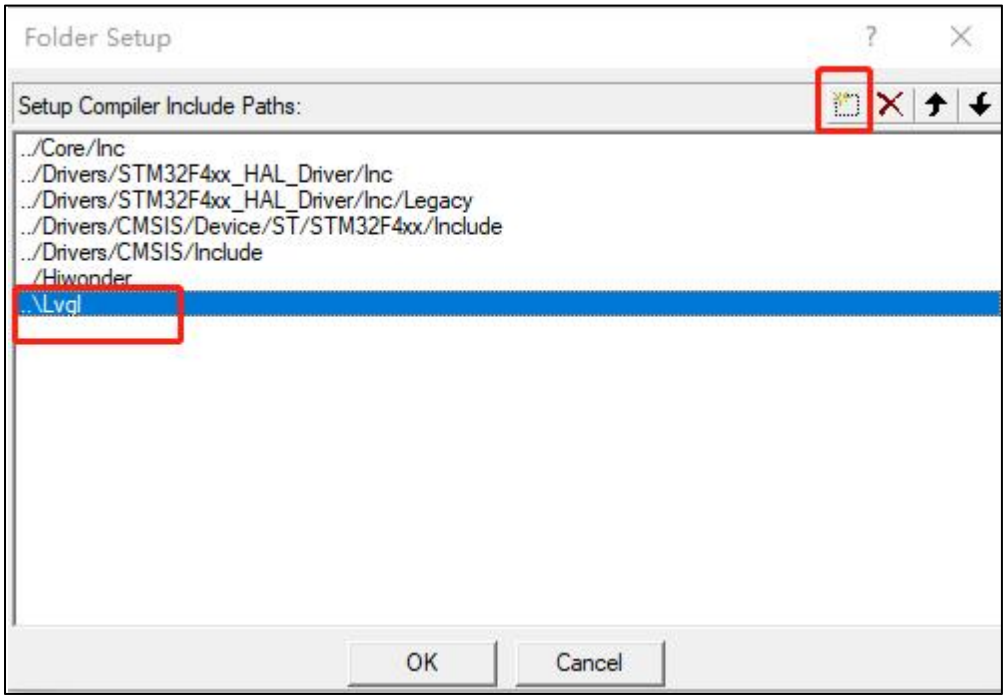
USE_HAL_DRIVER,STM32F407xx,

LV_CONF_INCLUDE_SIMPLE,

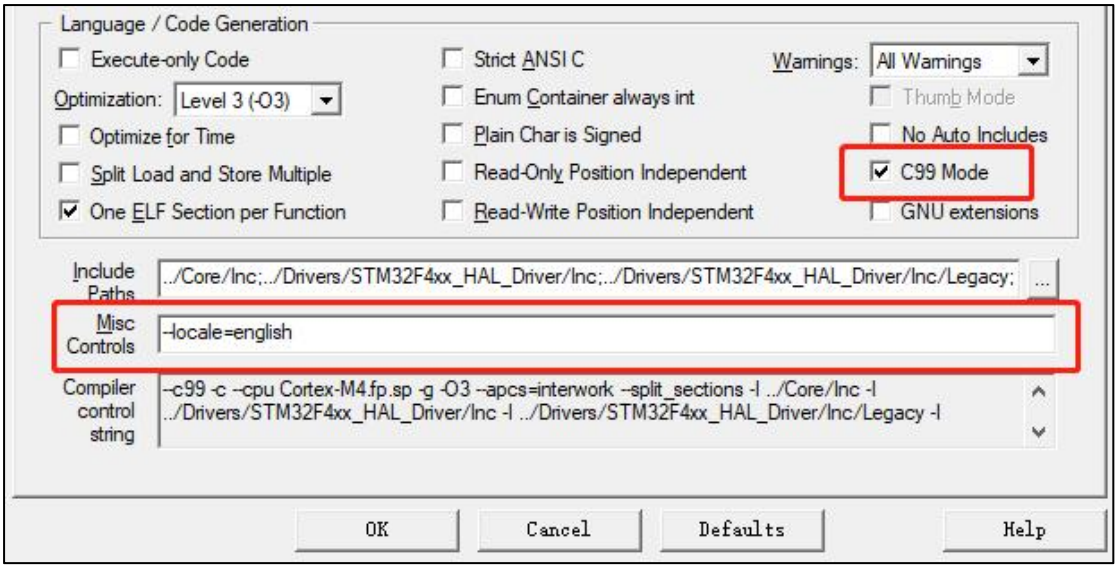
LV_LVGL_H_INCLUDE_SIMPLE,

NDEBUG

Then include the header files (Include Paths): Add the 'Lvgl' folder."



Set character matching recognition (--locale=english), and similarly, set the compilation mode to C99:



Back to the software interface, click on the 'Lvgl/porting.c' folder, as shown in the figure below:

Modify '#if 0' to '#if 1' in the 'lv_port_disp.c' file,

Change '#include lv_port_disp_template.h' to '#include lv_port_disp.h',

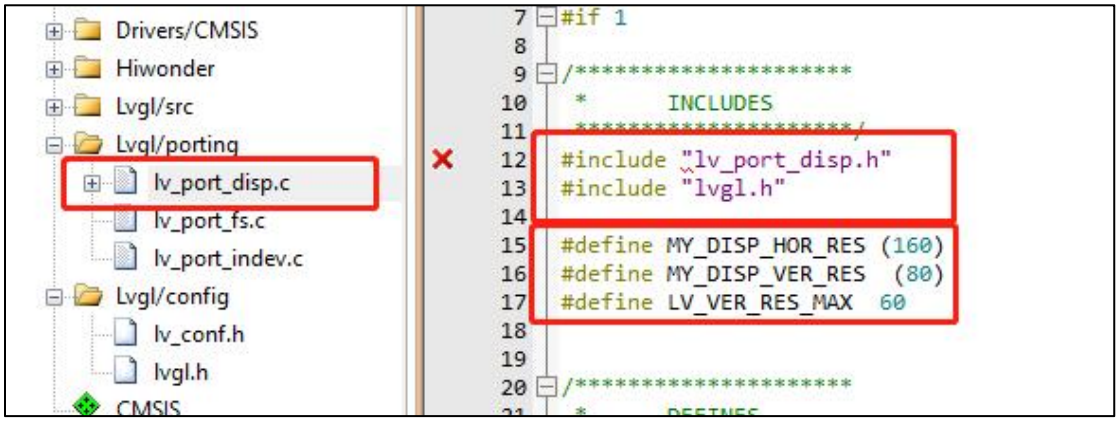
If there are red wavy lines and error messages, disregard them,

Then set the macro definitions for width and length according to your screen's resolution:

```
#define MY_DISP_HOR_RES (160)

#define MY_DISP_VER_RES    (80)

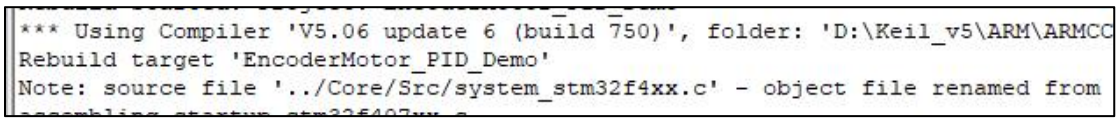
 #define LV_VER_RES_MAX    60
```



Click-on the compilation button for compilation:



Wait for the compilation to be completed:



If no errors occur (warnings can be ignored), it indicates that the compilation is complete, and lvgl has been successfully integrated into the project.

```
Build started: Project: Encoder_Demo
*** Using Compiler 'V5.06 update 6 (build 750)', folder: 'D:\Keil_v5\ARM\ARMCC\Bin'
Build target 'Lvgl'
Note: source file '..\Lvgl\src\extra\themes\mono\lv_theme_mono.c' - object file rena
linking...
Program Size: Code=13220 RO-data=644 RW-data=20 ZI-data=3196
FromELF: creating hex file...
"Encoder_Demo\Encoder_Demo.axf" - 0 Error(s), 0 Warning(s).
Build Time Elapsed:  00:00:02
```

Note: lvgl8 requires Keil5 version 5.38 or higher with armclang to compile correctly.

Here, it is advisable to use lvgl7 for compilation as much as possible.